Implementation of Finite Field Arithmetic & Elliptic Curve Arithmetic for Diffie-Hellman Key Exchange Setting

Implemented and Documented by

Subhradip Palchaudhuri

CrS2321

Indian Statistical Institute, Kolkata

Supervised By

Prof. Sabyasachi Karati

Indian Statistical Institute, Kolkata

December 2, 2024

Abstract

This document explains the Codes associated with it in a simplified manner. This Implementation project is done as part of our coursework of the 3rd Semester of M.Tech in Cryptology and Security program of 2023-25. Our goal is to build up a setting which can perform the Diffie-Hellman Key Exchange Protocol. All of the implementation works are in C language.

At first, the class unanimously chose a 256-bit prime number. Then modular arithmetic such as Addition, Subtraction and Multiplication were implemented in the field $\mathbb{F}_p$. After this, Barrett Reduction, Exponentiation are done in that field. Finally, an Elliptic Curve setting was taken with minimal cofactor to implement Point Addition and Point doubling with coordinates of the points being elements of our field $\mathbb{F}_p$. Then Scalar Multiplication of a point is implemented.

Contents

Abstract **1**

1 The Finite Field Paradigm **4**

1.1 The Prime p : 4

1.2 Arithmetic Operations in $\mathbb{Z}_p$: 4

2 The Codes **5**

2.1 Conversion (convert.h) 5

2.1.1 printVal( ) : 5

2.1.2 leftShift_32( ) & rightShift_32( ) : 5

2.1.3 Base Switch : 5

2.1.4 base_8_to_30( ) & base_30_to_8( ) 6

2.2 Arithmetic Operations (add_mul.h) 7

2.2.1 printVal_64( ) : 7

2.2.2 Addition [add( )] : 7

2.2.3 Common Subtraction [sub_com( )] : 8

2.2.4 Subtraction in $\mathbb{Z}_p$ [sub( )] : 9

2.2.5 Coordinate Subtraction in EC [sub4ecc( )] : 9

2.2.6 Multiplication [mul( )] : 10

2.3 Barrett Reduction (barrett.h) 11

2.3.1 Extended Multiplication [ext_mul( )] : 11

2.3.2 Shrinking [shrink( )] : 11

2.3.3 The Barrett Reduction [bar_red( )] : 13

2.4 Exponentiation (expo_mult.h) 14

2.4.1 Length of the Exponent [len( )] : 14

2.4.2 Left-to-Right Exponentiation [L2R_mult( )] : 15

2.4.3 Right-to-Left Exponentiation [R2L_mult( )] : 16
2.4.4 The Montgomery Ladder Exponentiation [montgomery-Ladder_mult( )] : 17
2.4.5 Constant-Time Montgomery Ladder Exponentiation [antiSC_montgomeryLadder_mult( )] : 18
2.5 Elliptic Curve Arithmetic (ecc_base.h) 18
2.5.1 printPoint( ) : 19
2.5.2 Equality Check [isEqual( )] : 19
2.5.3 Inverse [inv( )] : 19
2.5.4 Curve Satisfiability [inCurve( )] : 19
2.5.5 Point Addition [addition( )] : 19
2.5.6 Point Doubling [doubling( )] : 20
2.6 ECC Scalar Multiplication (ecc_scal_mult.h) 20
2.6.1 Left-to-Right Scalar Multiplication [ECC_L2R_mult( )] : . 21
2.6.2 Right-to-Left Scalar Multiplication [ECC_R2L_mult( )] : . 22
2.6.3 The Montgomery Ladder Scalar Multiplication [ECC_montgomeryLadder_mult( )] : 22
2.6.4 Constant-Time Montgomery Ladder Scalar Multiplication [ECC_antiSC_montgomeryLadder_mult( )] : 22

3 Testing and Evaluation **24**

Acknowledgements **25**

Chapter 1

The Finite Field Paradigm

1.1 The Prime p :

We choose the 256-bit prime p =105470615072424007464777057006017113535036866827082468263120632948849084329973 and fix it for this project. We know that $\mathbb{Z}_p = \{0, 1, ..., p-1\}$ is a field with respect to addition and multiplication, where each element represents the class of that element in $\mathbb{Z}$. We will work on this field.

1.2 Arithmetic Operations in $\mathbb{Z}_p$:

for $a, b \in \mathbb{Z}_p$ with $a \neq 0$,

- $a + b = a + b \ (mod\ p)$
- $a.b = ab \ (mod\ p)$
- $(-a)$ is the additive inverse of a in $\mathbb{Z}_p$. i.e. $a + (-a) = 0$
- a^{-1} is the additive inverse of a in $\mathbb{Z}_p$. i.e. $a.a^{-1} = 1$

Chapter 2

The Codes

We built the library for our functions in the header files, and use them in the respective .c files for testing purposes. Note that, in all of our implementation, we use *while* loop and *break* command inside it for executing *if-else operations*. This is done just to avoid the use of *if-else* commands in the codes.

2.1 Conversion (convert.h)

2.1.1 printVal( ) :

The **printVal_8** and **printVal_32** are functions to simply print an 8-bit and 32-bit integer array x of size n in decimal values.

2.1.2 leftShift_32( ) & rightShift_32( ) :

The **leftShift_32** and the **rightShift_32** function simply shifts a *uint32_t* integer x by n bits to the left and right respectively.

2.1.3 Base Switch :

We take the 256-bit prime p as our first input. Now, in C, we cannot input such a massive integer, as it goes beyond the range of even unsigned long integer. So, we must devise a way to take such a huge integer as an input.

We define a 32-element array of *uint8_t* variables, say $a[32]$. We then input the array elements in such a way that each element corresponds to an 8-bit chunk of the 256-bit value in *little endian* setting, i.e. the least significant 8 bits are stored in the lowest memory address of a. So, if $p[0-255]$ is the *big endian* binary expression of our 256-bit prime, then $a[0] = p[248-255],\ a[1] = p[240-247],\ \ldots,\ a[31] = p[0-7]$.

Thus we've successfully taken the prime input in *Base* 2^8, i.e. if $\theta = 2^8$, then $p = a[0]\theta^0 + a[1]\theta^1 + \ \ldots\ + a[31]\theta^{31}$. Equivalently, we can say that we have expressed p as a polynomial of θ with degree 31. But, performing arithmetic operations on such a long (high-degree) polynomial is cumbersome. So, we transform this 31-degree polynomial expression of $\theta = 2^8$ of p into a 9-degree polynomial expression of $\theta = 2^{30}$ using the function **base_8_to_30( )** as described below.

2.1.4 base_8_to_30( ) & base_30_to_8( )

base_8_to_30( ) :

In this function, we convert a *Base* 2^8 array for a 256-bit integer (such as array $a[32]$) into its *Base* 2^{30} version (say $a1[9]$ with *uint32_t* array elements) through bit manipulation. Each *uint32_t* element of $a1$ has the most significant 2 bits as 0's and $a1[8]$ (the highest address of $a1$) has the most significant 16 bits as 0's. Thus, we get a proper representation of a 256-bit positive integer through this format ($30 \times 8 + 16 = 256$). We construct this function by using a *for* loop with carefully chosen conditions and adjustments to fulfill the required task.

base_30_to_8( ) :

Here, we do the exact opposite of what we did in **base_8_to_30( )**. We convert the *Base* 2^{30} version with *uint32_t* array elements back to the *Base* 2^{30} version with array elements *uint8_t*.

2.2 Arithmetic Operations (add_mul.h)

At the very beginning, we define our 256-bit prime p globally in *Base* 2^{30} format, which we get as an output of the file **convert.c** with *Base* 2^8 input of our prime p. Now, for two elements $a, b \in \mathbb{Z}_p$ in *Base* 2^{30} representation, we'll compute the addition, subtraction and multiplication of the values. But, we'll have to take into account the fact that addition and multiplication of two 256-bit integers can result into a 257-bit and 512-bit integers respectively. So, we'll have to reduce these values so that it stays in our field $\mathbb{Z}_p$. We'll use **shrink( )** and **bar_red( )** functions to reduce the values in the **barrett.h** library. For now, we'll only do the arithmetic operations here in **add_mul.h** without the reduction.

2.2.1 printVal_64( ) :

This is purely used to check the values of some *uint64_t* arrays when we perform multiplication later in this library.

2.2.2 Addition [add( )] :

We apply the following algorithm in this function.

Algorithm 1 Addition : add( )

Input : uint32_t $a[9]$, $b[9]$
Output : uint32_t $c[9]$

$c[0] \leftarrow 0$
for $i = 0$ to 7 **do**
  $c[i] \leftarrow c[i] + (a[i] + b[i])$ ▷ To add with the existing Carry
  $c[i+1] \leftarrow c[i] >> 30$ ▷ To append the Overflow as Carry for the next chunk
  $c[i] \leftarrow (c[i] << 2) >> 2$ ▷ The CLICK to keep the value in *Base* 2^{30}
end for
$c[8] \leftarrow c[8] + (a[8] + b[8])$
return c

The symbols $x >> n$ and $x << n$ mean right shift and left shift of x in Binary representation by n bits, respectively.

2.2.3 Common Subtraction [sub_com( )] :

This is the common part of Subtraction we perform for both the Field Arithmetic and Elliptic Curve Arithmetic. The subtraction goes as below.

Algorithm 2 Common Subtraction : sub_com( )

Input : uint32_t $a[9]$, $b[9]$
Output : uint32_t $c[9]$

1: $c[0] \leftarrow 0$
2: **for** $i = 0$ to 7 **do**
3: **if** $a[i] \geq b[i]$ **then**
4: $c[i] \leftarrow a[i] - b[i]$
5: **else**
6: $a[i+1] \leftarrow a[i+1] - 1$ ▷ The Borrow from the next chunk $k \leftarrow i$
7: **while** $a[k+1] = -1$ & $(k+1) < 8$ **do** ▷ $-1 = 2^{32} - 1$ for uint32_t
8: $a[k+1] \leftarrow (a[k+1] << 2) >> 2$ ▷ The CLICK
9: $a[k+2] \leftarrow a[k+2] - 1$
10: $k++$
11: **end while**
12: $c[i] = (2^{30} + a[i]) - b[i]$ ▷ After the Borrow, $a[i]$ becomes $(2^{30} + a[i])$
13: **end if**
14: **end for**
15: **return** c

Again we emphasize the fact that we used *while-break* in our Codes instead of *if-else* statements to make them run in constant time.

In **sub_com**, We basically did the subtraction for the all the chunks except for the most significant chunk, while taking Borrow from it if needed. In the subsequent parts, we'll use this function to subtract in 2 different kind of settings to get 2 different kind of outputs (as we will require them).

In *line 7* of Algorithm 2, we enter the *while loop* iff the $(k+1)$-th chunk becomes -1 (i.e. $2^{32} - 1$ after wrapping around for *uint32_t*), which means the $(k+1)$-th chunk was 0 to begin with in that iteration. This is the case where $a < b$. Now, the same method cannot be done for the chunk $a[8]$ as we can't take Borrow from the next chunk in this case, for there is no chunk left to take Borrow from. We have to tackle this case slightly more carefully for gaining

optimum precision in the outcome.

2.2.4 Subtraction in $\mathbb{Z}_p$ [sub( )] :

This is the function we use for Field subtraction in $\mathbb{Z}_p$. The algorithm goes as follows.

Algorithm 3 Subtraction in $\mathbb{Z}_p$: sub( )

Input : uint32_t $a[9]$, $b[9]$
Output : uint32_t $c[9]$

```
sub_com(a, b, c)
a[8] ← (a[8] << 2) >> 2
c[8] ← a[8] − b[8]
c[8] ← (c[8] << 2) >> 2        ▷ To counter the case of c being negative.
return c
```

If $a >= b$ (i.e. $a[8] >= b[8]$), then it's pretty obvious that Algorithm 3 works fine. Now we don't need to take the case of $a < b$ into consideration as we'll not need to subtract a bigger field element from a smaller one in our Field Arithmetic (later we'll see why). So, we are okay with this Algorithm.

2.2.5 Coordinate Subtraction in EC [sub4ecc( )] :

While implementing the Elliptic Curve setting, we'll need a different kind of subtraction under our field. The Algorithm goes as follows.

Algorithm 4 Coordinate Subtraction : sub4ecc( )

Input : uint32_t $a[9]$, $b[9]$
Output : uint32_t $c[9]$

```
sub_com(a, b, c)
c[8] ← a[8] − b[8]
if c[8] ≥ p[8] then
    a1 ← add(c, p)
    c ← a1
end if
return c
```

In $\mathbb{Z}_p$, $-x = -x+p \ (mod\ p)$. In Elliptic Curve Setting, when we subtract a bigger coordinate from a smaller one, the most significant chunk value ($c[8]$) wraps around and satisfies $c[8] >= 2^{32} - p[8]$, thus $c[8]$ becomes way bigger than $p[8]$. Then, we just add p to the resulting value to get back into a valid *Base* 2^{30} representation, which is in $\mathbb{Z}_p$. This is exactly what we do in the *if condition* in *line 3* of Algorithm 4.

Now, question may arise as to why we need to define these 2 subtractions seperately. We'll discuss this later in the *Barrett Reduction* Section.

2.2.6 Multiplication [mul( )] :

The Algorithm goes as below.

Algorithm 5 Multiplication : mul( )

Input : uint32_t $a[9]$, $b[9]$
Output : uint32_t $d[18]$

1: uint64_t $m[17]$
2: **for** $i = 0$ to 16 **do**
3: $\quad m[i] \leftarrow \sum_{k+j=i,\ 0 \leq k,j \leq 8} a[k].b[j]$
4: **end for**
5: $d[0] \leftarrow lsb_{30}(m[0])$
6: $d[1] \leftarrow (m[0] >> 30) + lsb_{30}(m[1])$
7: $C \leftarrow Carry(d[1])$
8: $d[1] \leftarrow (d[1] << 2) >> 2$
9: **for** $i = 2$ to 16 **do**
10: $\quad d[i] \leftarrow C + (m[i-2] >> 60) + (m[i-1] >> 30) + lsb_{30}(m[i])$
11: $\quad C \leftarrow Carry(d[i])$
12: $\quad d[i] \leftarrow (d[i] << 2) >> 2$
13: **end for**
14: $d[17] \leftarrow C + (m[15] >> 60) + (m[16] >> 30)$
15: **return** d

From Algorithm 5 itself, it's pretty convenient what the function does. It first multiplies a & b using the methods of polynomial multiplication to get m which is an array of *uint64_t*. Then it manipulates the bits of each element of m to produce the multiplication value d in our *Base* 2^{30} representation.

2.3 Barrett Reduction (barrett.h)

Clearly, the output we got from **mul( )** in the previous section results in a 512-bit integer, which is definitely not in $\mathbb{Z}_p$. So, we must reduce it to an element of $\mathbb{Z}_p$. We'll use a special efficient method of reduction here, namely *The Barrett Reduction*.

First, we declare a pre-computed global value T in *Base* 2^{30} format for our convenience. Let $\theta = 2^{30}$, then

$$T = \lfloor \frac{\theta^{2L}}{p} \rfloor$$

where L = total number of chunks in Base θ representation of $p = 9$ (in our case).

Notice that, $\theta^{2L} = 2^{30\times2\times9} = 2^{540}$. So, 2^{540} is divided by a 256-bit integer (p) to yield T, which is roughly (540-256 =) 284 bits long. To represent T in *Base* 2^{30} format, we'll need 10 chunks, 1 more than we're used to in our setting so far. Hence, $T[10]$ is declared.

2.3.1 Extended Multiplication [ext_mul( )] :

We'll need to multiply a specific value with T in the Barrett Reduction. But, as we have defined our **mul( )** function, it only applies on integers which are 9-chunk long. So, here we have to define an Extended version of **mul( )** which can be applied on T. The algorithm (Algorithm 6) is almost similar to Algorithm 5.

2.3.2 Shrinking [shrink( )] :

If we end up with a value that's slightly bigger than p, we use this function to get it back inside $\mathbb{Z}_p$.

Basically, Algorithm 7 subtracts p from x until it becomes less than of equal to p. But, implementing such a function in our *Base* 2^{30} setting is harder than what it looks like in the Algorithm.

First, we need to check if $x[8] \geq p[8]$. That's what we did in the first *while loop* in the code. But, this doesn't guarantee if x is indeed bigger than p.

Algorithm 6 Extended Multiplication : ext_mul( )

Input : uint32_t $a[10],\ b[10]$
Output : uint32_t $d[20]$

```
uint64_t m[19]
for i = 0 to 18 do
    m[i] ← Σ_{k+j=i, 0≤k,j≤9} a[k].b[j]
end for
d[0] ← lsb_30(m[0])
d[1] ← (m[0] >> 30) + lsb_30(m[1])
C ← Carry(d[1])
d[1] ← (d[1] << 2) >> 2
for i = 2 to 18 do
    d[i] ← C + (m[i − 2] >> 60) + (m[i − 1] >> 30) + lsb_30(m[i])
    C ← Carry(d[i])
    d[i] ← (d[i] << 2) >> 2
end for
d[19] ← C + (m[17] >> 60) + (m[18] >> 30)
return d
```

Algorithm 7 Extended Multiplication : shrink( )

Input : uint32_t $x[9],\ p[9]$
Output : uint32_t $x[9]$

```
while x >= p do
    x ← x − p
end while
return x
```

To complete our check for $x \geq p$, we then take 2 variables; a t for the chunk index going from the most significant towards the least significant one in search of the first chunk of x which is bigger than the corresponding chunk of p. And we have a $flag$ to mark that chunk-index. If no such chunk exist, i.e. $x = p$, then we have $flag = 0$ and we subtract p from x to essentially get 0. If $x < p$, then we go straight in the last *while loop* and fix the $flag = 100$ (an arbitrarily large value) to get out of the function with x being what it initially was. Otherwise, we subtract and "shrink" x until it gets inside $\mathbb{Z}_p$

2.3.3 The Barrett Reduction [bar_red( )] :

Now, we'll explain the reduction's algorithm.

Algorithm 8 Barrett Reduction : bar_red( )

Input : uint32_t $d[18]$
Output : uint32_t $x[9]$

1: $Q \leftarrow \lfloor \frac{d}{\theta^{L-1}} \rfloor$ ▷ $\theta = 2^{30}$, $L = 9$
2: $QT \leftarrow ext_mul(Q, T)$
3: $Q1 \leftarrow \lfloor \frac{QT}{\theta^{L+1}} \rfloor$
4: $Qp \leftarrow mul(Q1, p)$
5: $Qp1 \leftarrow Qp \ (mod\ \theta^L)$
6: $d1 \leftarrow d \ (mod\ \theta^L)$ ▷ In our setting, taking (mod θ^L) suffices
7: $x \leftarrow sub(d1, Qp1)$
8: $x \leftarrow shrink(x, p)$
9: **return** x

Typically, it is evaluated that for $d = qp + x$, with $0 \leq x \leq (p - 1)$,

$$q - 2 \leq Q1 \leq q$$

where $Q1$ is evaluated in *line 3* of Lgorithm 8. Hence, we need to perform the *while loop* in the **shrink( )** function at most twice. This is quite efficient. Also, as $Q1$ never goes beyond q, it means in *line 4*, Qp is never bigger than d, and hence in *line 7*, x never becomes negative. That's why we did not define subtraction of a bigger value from a smaller one in our setting.

Notice that, dividing by d by $\theta^8 = 2^{240}$ and taking the floor of it essentially means right shifting d by 240 bits. And taking $Qp1$ (mod $\theta^L = 2^{270}$) means taking the LSB 270 bits of $Qp1$.

Also, with respect to bits, $|T| \approx 284$, $|Q| \approx 512-240 = 272 \implies$ —QT—$\approx 272 + 284 = 556$ and $|Q1| \approx 556 - 300 = 256$. Thus, taking (mod $\theta^L = 2^{270}$) suffices here.

Now also notice that for a value x which is slightly bigger than p, if we had used the method we used for **sub4ecc** in 2.2.5, it would keep on adding p to x until reaching something $\geq 2^{32}$ to be able to wrap it around, which will take massive time. So, shrinking with **sub( )** helps here. On the contrary, if we use **shrink( )** for ECC point subtraction, the out-of-range coordinate (close to 2^{32}) will subtract p until reaching within the value of p, which will again take huge amount of time. This is why we defined the 2 subtractions seperately to compute efficiently.

2.4 Exponentiation (expo_mult.h)

Our goal is to setup an environment to perform the Diffie-Hellman Key Exchange Protocol. For that, we need to implement exponentiation in our field $\mathbb{Z}_p$. And even before that, we need to find a generator of the field $\mathbb{Z}_p$. We have used sage codes to find such a generator. In **expo_mult.c**, we have used $g = 2$ as a generating base for testing out the code.

Note that here we have implemented 4 algorithms of exponentiation, and the first 3 of them use *if-else* commands and the 4th one does not. For Constant-Time implementation, we'll use the 4th one. So, we haven't modified the rest of the algos and kept them as they are with *if-else* commands. The functions used in this code are as follow.

2.4.1 Length of the Exponent [len( )] :

This is to evaluate the binary length of the exponent. This value will be pivotal in performing the exponentiations. For the convenience of user who'll input an exponent value n, we consider it to be an array of *uint8_t*. So, for a 256-bit value of n, the array has at most 32 elements. Algorithm 9 explains it.

Algorithm 9 Length : len( )

Input : uint8_t $n[32]$
Output : int l

$n1 \leftarrow base_8_30_(n)$
$n1 \leftarrow shrink(n1, p)$ ▷ Shrinking if $n \notin \mathbb{Z}_p$
$n \leftarrow base_30_8_(n1)$
$i \leftarrow 31$
while $n[i] == 0$ **do**
    $i - -$
end while
$a \leftarrow n[i]$, $k \leftarrow 8$
while $a < 128$ **do**
    $a \leftarrow a << 1$
    $k - -$
end while
$l \leftarrow (i \times 8) + k$
return l

2.4.2 Left-to-Right Exponentiation [L2R_mult( )] :

We go from the MSB to the LSB of the exponent in this method and multiply the base value g with itself accordingly. Algorithm 10 describes the method.

Algorithm 10 Left-to-Right Exponentiation : L2R_mult( )

Input : uint32_t $g[9]$, uint8_t $n[32]$
Output : uint32_t $h[9]$

$h \leftarrow 1$
$l \leftarrow len(n)$
for $i = l - 1$ to 0 **do**
    $h \leftarrow h^2$
    if $n_i == 1$ **then** ▷ n_i = the i-th bit of n in little endian setting
        $h \leftarrow h.g$ ▷ Barrett Reduction necessary while implementing
    end if
end for
return h

Correctness :

$$n = n_{l-1}2^{l-1} + n_{l-2}2^{l-2} + \dots n_1.2 + n_0$$

$$= (\dots(n_{l-1}.2 + n_{l-2})2 + n_{l-3})2 + \dots + n_2)2 + n_1)2 + n_0$$

$$\implies g^n = g^{(\dots(n_{l-1}.2+n_{l-2})2+n_{l-3})2+\dots+n_2)2+n_1)2+n_0}$$

Clearly, in step $k-1$, the expression is $[\dots n_k)2 + n_{k-1})2 + n_{k-2}]$ and g is multiplied to h iff $n_{k-1} = 1$, but it gets squared in either case. So depicts the Algorithm.

2.4.3 Right-to-Left Exponentiation [R2L_mult( )] :

We go from the LSB to the MSB of the exponent in this method and multiply the base value g with itself accordingly. Algorithm 11 describes the method.

Algorithm 11 Right-to-Left Exponentiation : R2L_mult( )

Input : uint32_t $g[9]$, uint8_t $n[32]$
Output : uint32_t $h[9]$

```
h ← 1
l ← len(n)
for i = 0 to l − 1 do
    if n_i == 1 then        ▷ n_i = the i-th bit of n in little endian setting
        h ← h.g             ▷ Barrett Reduction necessary while implementing
    end if
    g ← g^2
end for
return h
```

Correctness :

$$g^n = g^{n_0 + n_1 2 + n_2 2^2 + \dots + n_{l-1}2^{l-1}}$$

$$= \left(g^{2^0}\right)^{n_0} \times \left(g^{2^1}\right)^{n_1} \times \left(g^{2^2}\right)^{n_2} \times \dots \times \left(g^{2^{l-1}}\right)^{n_{l-1}}$$

So, in each step g gets squared. Only if $n_k = 1$ at k-th step, the resulting multiplied value gets included in the result h. So, the algorithm is correct.

2.4.4 The Montgomery Ladder Exponentiation [montgomeryLadder_mult( )] :

This is a very special way to implement the Exponentiation. The Algorithm goes as follows.

Algorithm 12 Montgomery Ladder Exponentiation : montgomeryLadder_mult( )

Input : uint32_t $g[9]$, uint8_t $n[32]$
Output : uint32_t $S[9]$

```
S ← g
R ← g^2
l ← len(n)
for i = l − 2 to 0 do
    if n_i == 0 then        ▷ n_i = the i-th bit of n in little endian setting
        R ← S.R
        S ← S^2             ▷ Barrett Reduction necessary while implementing
    else
        S ← S.R
        R ← R^2
    end if
end for
return S
```

Correctness :

We'll prove the correctness by Induction. Let the hypothesis be

At step $l-i-1$, $N_{i+1} = (n_{l-1}n_{l-2}\ldots n_{i+1})_{10}$. Then $S = g^{N_{i+1}}$ & $R = g^{N_{i+1}+1}$

$len(n) = l \implies n_{l-1} = 1$. Hence, at step $i = 0$, $N_1 = (n_{l-1})_{10} = 1$. And $S = g = g^1$ & $R = g^2 = g^{1+1}$. So, the hypothesis holds for $i = 0$.

Let for $i = l-i-1$, it's true. So, $N_{i+1} = (n_{l-1}n_{l-2}\ldots n_{i+1})_{10}$ and $S = g^{N_{i+1}}$ & $R = g^{N_{i+1}+1}$.

So, $N_i = (n_{l-1}n_{l-2}\ldots n_{i+1}n_i)_{10}$

If $n_i = 0$, then $N_i = 2N_{i+1}$. So, $S_i = S_{i+1}^2 = (g^{N_{i+1}})^2 = g^{2N_{i+1}} = g^{N_i}$

And $R_i = S_{i+1} \times R_{i+1} = g^{N_{i+1}} \times g^{N_{i+1}+1} = g^{2N_{i+1}+1} = g^{N_i+1}$

And if $n_i = 1$, then $N_i = 2N_{i+1} + 1$. So, $S_i = S_{i+1} \times R_{i+1} = g^{N_{i+1}} \times g^{N_{i+1}+1} = g^{2N_{i+1}+1} = g^{N_i}$
And $R_i = R_{i+1}^2 = (g^{N_{i+1}+1})^2 = g^{2N_{i+1}+2} = g^{N_i+1}$

So, the hypothesis holds.

2.4.5 Constant-Time Montgomery Ladder Exponentiation [antiSC_montgomeryLadder_mult( )] :

In this function, we'll implement the Montgomery Ladder Method in such a way that we won't need any *if-else* condition. So, this becomes a Constant-Time function that is protected against Side-Channel Attacks. The Algorithm is given in Algorithm 13.

Algorithm 13 Constant Time Montgomery Ladder Exponentiation : antiSC_montgomeryLadder_mult( )

Input : uint32_t $g[9]$, uint8_t $n[32]$
Output : uint32_t $S[9]$

Take $R_0,\ R_1$
$R_0 \leftarrow g$
$R_1 \leftarrow g^2$
$l \leftarrow len(n)$
for $i = l - 2$ to 0 **do**
    $R_{1-n_i} \leftarrow R_0.R_1$ ▷ n_i = the i-th bit of n in little endian setting
    $R_{n_i} \leftarrow R_{n_i}^2$ ▷ Barrett Reduction necessary while implementing
end for
$S \leftarrow R_0$
return S

2.5 Elliptic Curve Arithmetic (ecc_base.h)

The Equation $Y^2 = X^3 + aX + b$ typically represents an Elliptic Curve. And we can extend our Finite Field $\mathbb{Z}_p$ into an Elliptic Curve group defined on a specific curve. We've initially declared a & b globally to fix such a curve for our work. We have chosen a & b in such a way that the Cofactor upon doing quotient on $\mathbb{Z}_p$ becomes 1. So, we have the full freedom to work on the curve while taking points with coordinates spanning onto whole of the field.

Also, we'll need inverse of some elements of the field. From Fermat's Little Theorem, for any prime p, $a \in \mathbb{Z}_p \implies a^{p-1} = 1 \ (mod\ p) \implies a^{p-2}.a = 1 \ (mod\ p)$. So, a^{p-2} is the inverse of a in $\mathbb{Z}_p$. And we can use anyone of the exponentiation functions discussed in the previous section to find this value. That's exactly what's done in the **inv( )** function. And for this, we've declared $p2 = p - 2$ globally.

We define a *structure* **point** that holds 2 field elements as x and y-coordinates in *Base* 2^{30} format. This is done for simplicity.

2.5.1 printPoint( ) :

To print a point in P in P.x : P.y format in *base* 2^{30}.

2.5.2 Equality Check [isEqual( )] :

By checking chunk by chunk correspondingly for 2 field elements a & b, it checks if the elements are equal or not. It returns 1 if $a = b$, and returns 0 if $a \neq b$.

2.5.3 Inverse [inv( )] :

Already discussed in the beginning of this Section.

2.5.4 Curve Satisfiability [inCurve( )] :

For our specified curve, this function takes a point P as input and checks if P is in our curve or not; returns 1 if P is in the curve, returns 0 otherwise.

The function simply computes whether the coordinates of the point satisfies the equation of the curve or not.

2.5.5 Point Addition [addition( )] :

We apply some well-known and popularly used Elliptic Curve characteristics and formulas to derive the addition of 2 points on the Curve. We have omitted the check of **inCurve( )** here as we have checked it while testing in **ecc_base.c**.

Checking it only once is enough, otherwise when we'll call this function multiple times, the checking will cost unnecessary time. So, here we assume that P is indeed a point on the curve. Also, we must make sure that $P \neq \pm Q$, where $-Q(x, y) = (x, -y)$, otherwise Addition on the points are undefined. We omit this check in the library too and put it while testing to save time.

Algorithm 14 Point Addition : addition( )

Input : point $P(P_x, P_y)$, point $Q(Q_x, Q_y)$
Output : point $R(R_x, R_y)$

1: $R_x \leftarrow \left(\frac{Q_y - P_y}{Q_x - P_x}\right)^2 - (P_x + Q_x)$
2: $R_y \leftarrow \left(\frac{Q_y - P_y}{Q_x - P_x}\right)(P_x - R_x) - P_y$
3: **return** R

2.5.6 Point Doubling [doubling( )] :

In this case too, we have to check for **inCurve( )**. Additionally, we'll have to make sure that the point P's (which we want to apply the function on) y-coordinate must not be 0, otherwise doubling is undefined. Here again we have omitted the checks as we have done that while testing.

Algorithm 15 Point Doubling : doubling( )

Input : point $P(P_x, P_y)$
Output : point $R(R_x, R_y)$

1: $R_x \leftarrow \left(\frac{3{P_x}^2 + a}{2P_x}\right)^2 - 2P_x$
2: $R_y \leftarrow \left(\frac{3{P_x}^2 + a}{2P_x}\right)(P_x - R_x) - P_y$
3: **return** R

2.6 ECC Scalar Multiplication (ecc_scal_mult.h)

If we notice carefully, we can find similarities between some of the Field Arithmetic Operations and Elliptic Curve Operations. For example, what **mul(**

) & **bar_red( )** does in our finite field, can be replicated by **addition( )** & **doubling( )** in Elliptic Curve Setting. Using this concept, we can replicate the Exponentiation done in our field $\mathbb{Z}_p$ in our Elliptic Curve Environment as Scalar Multiplication. This too will give an efficient Diffie-Hellman Key Exchange Protocol. The Algorithms are somewhat similar to the Exponentiation Algorithms. The Correctness of all of them follow exactly similarly (We just have to do our computations from Exponents to Multiplicands here). Let's have a look, shall we?

2.6.1 Left-to-Right Scalar Multiplication [ECC_L2R_mult( )] :

Algorithm 16 Left-to-Right Scalar Multiplication : ECC_L2R_mult( )

Input : point P, uint8_t $n[32]$
Output : point Q

```
Q ← P
l ← len(n)
for i = l − 2 to 0 do
    Q ← 2Q
    if n_i == 1 then        ▷ n_i = the i-th bit of n in little endian setting
        Q ← P + Q
    end if
end for
return Q
```

We must focus on one thing, though. The identity of the Elliptic Curve group, which is the point at infinity, is not fixed in EC Setting. It's not a precise point, rather a vague idea. So, we cannot initialize Q as the identity as we did with $h(= 1)$ for Exponentiation. That is why we rather consider the scalar n in such a way that after the first step, Q becomes P. E.g. for L2R method, we take the MSB of $n = 1$, and for R2L method, we take the LSB of $n = 1$. Whereas, for Montgomery Ladder Step, we take again the MSB to be 1, which is trivial as we computed length of n in such a way that the l-th bit of n is indeed 1.

2.6.2 Right-to-Left Scalar Multiplication [ECC_R2L_mult( )] :

Algorithm 17 Right-to-Left Scalar Multiplication : ECC_R2L_mult( )

Input : point P, uint8_t $n[32]$
Output : point Q

```
Q ← P
l ← len(n)
for i = 1 to l − 1 do
    if n_i == 1 then        ▷ n_i = the i-th bit of n in little endian setting
        Q ← P + Q
    end if
    P ← 2P
end for
return Q
```

2.6.3 The Montgomery Ladder Scalar Multiplication [ECC_montgomeryLadder_mult( )] :

See Algorithm 18

2.6.4 Constant-Time Montgomery Ladder Scalar Multiplication [ECC_antiSC_montgomeryLadder_mult( )] :

See Algorithm 19

Algorithm 18 Montgomery Ladder Scalar Multiplication : ECC_montgomeryLadder_mult( )

Input : point P, uint8_t $n[32]$
Output : point S

```
S ← P
R ← 2P
l ← len(n)
for i = l − 2 to 0 do
    if n_i == 0 then        ▷ n_i = the i-th bit of n in little endian setting
        R ← S + R
        S ← 2S
    else
        S ← S + R
        R ← 2R
    end if
end for
return S
```

Algorithm 19 Constant Time Montgomery Ladder Scalar Multiplication : ECC_antiSC_montgomeryLadder_mult( )

Input : point P, uint8_t $n[32]$
Output : point R_0

```
Take point R_0, R_1
R_0 ← P
R_1 ← 2P
l ← len(n)
for i = l − 2 to 0 do
    R_{1−n_i} ← R_0 + R_1        ▷ n_i = the i-th bit of n in little endian setting
    R_{n_i} ← 2R_{n_i}
end for
return R_0
```

Chapter 3

Testing and Evaluation

In the **.c** files, we have tested all the codes for various presumably random test values, and all the results match with the evaluation done by SageMath. This concludes the project.

Acknowledgements

- Prof. Sabyasachi Karati, The project guide.